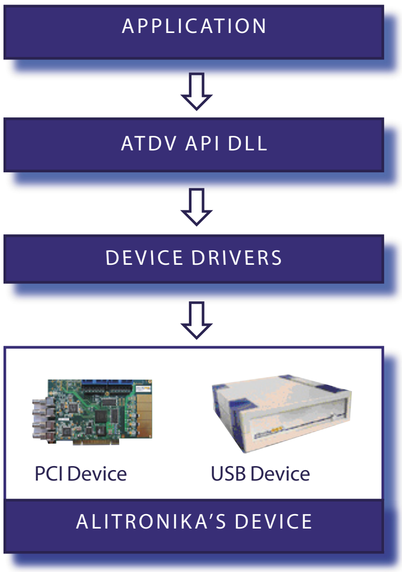
# PROGRAMMING INTERFACE FOR ALITRONIKA DVS DEVICES

**ATDV API**, stands for **A**li**T**ronika **D**igital **V**ideo **A**pplication **P**rogramming **I**nterface.



API Programming structure

**Writing programs using the ATDV API has the following advantages:**

- Hardware isolation.
- Device type independence.
- Compatible with both PCI and USB versions of the devices.
- Easy to use software interface.
- C++ Object oriented models.

The API is implemented in a DLL which could easily be replaced or upgraded. The C++ header file and the 'import-library' are used.

## The Object structure

The ATDV API allows application programmers to access the functions of Alitronika's device(s), without implementing any device driving. A device is represented as a C++ object that could be controlled.
There is one single CAtBoardManager object throughout the entire application.
This object is retrieved using CAtBoardManager::Instance(). The CAtBoardManager provides the application with access to the list of available devices.
The CAtBoard object of a specific device is retrieved with CAtBoardManager::GetBoard().

The CAtBoard object is used by the application to access the device's registers, decive configuration (e.g. Tuner functions) and the start/stop data transfers.
The reference documentation may be used to help during implementation of an application program

## ATDV_API Usage

The Compiler/linker needs to be setup to include the header file 'ATDV_API.H' and the 'ATDV_API.LIB' for static linking. The API code has been compiled with 'Multithreaded'. The application project must set this option to allow the use of background threads during playing and recording of transport streams.
When running the application, 'ATDV_API.DLL' must be in the system's PATH (normally installed in the Windows\system32 directory).

For Microsoft Visual C++ , the following steps should be taken:

- Copy all file ATDV_API.H and ATDV_API.LIB to the application development directory.
- Copy the ATDV_API.DLL to the Windows\system 32 directory.
- Open the application project as active project.
- Include the ATDV_API.LIB in the project.
- Click Project | Settings.
- On the C++ Tab, select: Category 'Code generation'.
- Select: Use run-time library 'Multithreaded".
- Close with the OK button.

The reference documentation is in the help files Atdv_api.chm and Atdv_api.chi. This is a searchable help system with a convenient browsing facility.

### Locating the device

The list of Alitronika devices installed can be retrieved through the CAtBoardManager. The device is selected from the list and a pointer to it is obtained with GetDeviceInfo(&iDevice).

See the code listing below

```
void PrintDev(char *p)
{
   cout << p << endl;
}

CAtDeviceList devlist;
ATDEVICEINFO iDevice;

CAtBoardManager *pMan = CAtBoardManager::
Instance();
pMan->GetDeviceList(devlist);

std::for_each(devlist.Names.begin(), devlist.
Names.end(), PrintDev);

CATBoard *pBoard = pMan->GetBoard(devlist.
Names[0]);
pBoard->GetDeviceInfo(&iDevice);

...
```

The PrintDev function is called for each device with the device's driver-id string. In this example, pBoard is initialized to the first device name found.

### Controlling a device

```
CAtRegisters RegAcc(pBoard);
ATREGISTRY &Registers = RegAcc;
DWORD &RecordConfig=
        RegAcc.m_RecordConfig;
DWORD &PlayConfig =
        RegAcc.m_PlayConfig;

TheATBoard.UpdateRegisters;

RecordConfig  |= AT_RCONF_DVB;


TheATBoard.UpdateRegisters;
```

The functions of each device are controlled be writing to the appropriate register in the Firmware. These registers are made available for reading and writing so the the programmer could directly control the device and its fuction.
The CatRegisters object is used to access the registers, while a lock is maintained. The lock is released when the object goes out of scope (is destroyed). In this way, multiple threads of execution can safely access the registers.

### Bitrate issues

The bitrate for the output stream is written into the bitrate register. The bitrate of the input stream is calculated by the Firmware. The Packet Pulse Counter register holds the bitrate as number of clock pulses for 100 concecative transport packets. The CATBoard::GetInputBitrate() computes the bitrate, using a sophisticated filter to enhance accuracy. The filter must be initialized with CATBoard::InitInputBitrate().
When loop-through is active, the output bitrate register must periodically be updated (i.e. every 50ms) with the input bitrate. This ensures that the loop output can follow the input stream, without leading or lagging the input.

```
#define STATUS_TIMER  60
#define STATUS_MS     50

BOOL CmyView::OnInitDialog()
{
 ...
 pBoard->InitBitrateFilter(STATUS_MS);
 SetTimer(STATUS_TIMER, STATUS_MS, 0);
}


void CMyView::OnTimer(UINT nIDEvent)
{
 ...
 int Bitrate =
      pBoard->GetInputBitrate();
 m_Str.Format("%.2f", Bitrate/1e6);
 m_BrStatic.SetWindowText(m_Str);
}
```

### Recording and Playing Transport stream

The record and play functions of the Alitronika devices provide both hardware and software buffering. Although this relaxes the timing requirement of the application, the application (or the record/play threads) must run at a higher priority.
The examples for record and play are given in the ATDV_API help file.
The functions involved are members of CATBoard: StartPlaying(), StopPlaying(), SendPlayPacketDirect(), StartRecording(), StopRecording(), GetRecordPacketDirect().

## Controlling the device's data source

A data source is the input of the Alitronika devices processing core.
In most cases it is a tuner that needs configuration settings. The setting commands run through CATBoard::DvbSource().

```
...

/*
 *  Tune the Satellite tuner to the
 *  frequency and set the
 *  transponder's Symbol rate.
 */
void SetDVBsourceStation
  (u32 nFreq, u32 nSymbRate)
{
    dvb_frontend_parameters parms;

    parms.frequency = nFreq;
    parms.inversion = INVERSION_AUTO;

    parms.u.qpsk.fec_inner =
        FEC_AUTO;
    parms.u.qpsk.symbol_rate =
        nSymbRate;

    pBoard->DvbSource
       (SRC_SET_FRONTEND, &parms);
}
```

# THE REGISTER SETTINGS FOR ALL ALITRONIKA DEVICES

## OPERATIONAL REGISTERS

The operational registers of the Alitronika devices are listed below. Since all register accesses are 32 bit transfers, the operational registers are grouped into four 8-bit registers.

- **Input Configuration Registers (Record)**

Record_Config Address = H"00"

| Bit | Name | Function | Level |
|---|---|---|---|
| 0 | DVB | Selects DVB Mode | High = Enabled |
| 1 | SMP | Select SMPTE Mode | High = Enabled |
| 2 | SER | Select Serial input | High = Enabled |
| 3 | PAR | Select Parallel input | High = Enabled |
| 4 | LEN | Input Loop through Enable | High = Enabled |
| 5 | LRC | Input Loop through re-clocked Enable | High = Enabled |
| 6 | IIO | Input I/O Processing | High = Enabled |
| 7 | FEN | Fly wheel enable | High = Enabled |
| 8 | ETS | Enable Time Stamping | High = Enabled |
| 9 | PID | Enabling PID filtering | High = Enabled |
| 10 | TEX | PID Table Exclusive | High = Exclusive Table |
| 11 | MUT1 | Mute Audio1 | High = Mute |
| 12 | MUT2 | Mute Audio2 | High = Mute |
| 23:13 | RSV | Reserved (not use in current design) | Normally Low |
| 24 | LNBP | Enable LNB | High = Enabled |
| 25 | VSEL | Enable extended LNB Voltage | High = Enabled |
| 26 | LLC | Enable cable voltage drop compensator | High = Enabled |
| 27 | H/V | Select Horizontal or Vertical | High = Horizontal |
| 28 | ENT | Enable internal 22 KHz modulator | High = Enabled |
| 31:29 | RSV | Reserved (not use in current design) | Normally Low |

- **Output Configuration Registers (Play)**

Play_Config Address = H"04"

| Bit | Name | Function | Level |
|---|---|---|---|
| 0 | DVB | Selects DVB Mode | High = Enabled |
| 1 | SMP | Select SMPTE Mode | High = Enabled |
| 2 | SER | Select Serial output | High = Enabled |
| 3 | PAR | Select Parallel output | High = Enabled |
| 4 | 188 | Select 188 TP Size | High = Enabled |
| 5 | 204 | Select 204 TP Size | High = Enabled |
| 6 | 188+16 | Add 16bytes to188TP | High =188+16 Mode |
| 7 | Raw | Send Raw Data Out | High =Enabled |
| 8 | CLK27 | Select 27MHz Clock for output | High = Clk27 Selected |
| 9 | EXCLK | Select External Clock for output | High = Ex Clock |
| 10 | BURST | Burst Mode TP | High = Enabled |
| 11 | GTP | Hardware Generated TP | High = Enabled |
| 12 | NTP | Null Packet | High = Null TP |
| 13 | CTP | Counter TP | High = Counter TP |
| 14 | BLK | Blanking | High=Enabled |
| 15 | IOP | Output I/O Processing | High=Enabled |
| 16 | TRS | Detect TRS | High=Enabled |
| 17 | SPT | Enable Serial Pass-Through output | High=Enabled |
| 18 | PPT | Enable Parallel Pass-Through output | High=Enabled |
| 19 | CSP | Serial to Parallel Conversion | High=Enabled |
| 20 | CPS | Parallel to Serial Conversion | High=Enabled |
| 31:22 | RSV | Reserved (not use in current design) | Normally Low |

- **Bit Rate Registers (Play & Monitor)**

Bit Rate Address = H"08"

| Bit | Name | Function | Level |
|---|---|---|---|
| 31:0 | Bit Rate | Bit Rate | |

- **Status& Error Registers (Play & Record)**

Status_Error Address = H"0C"

| Bit | Name | Function | Level |
|---|---|---|---|
| 0 | R_TPS | Input TP Size | High =188, Low = 204 |
| 1 | R_PSE | Packet Size Error (not 188 or 204) | High = Error |
| 6:2 | VER | Firmware version | See Firmware version Table |
| 7 | BUS | Bus type indication | High = PCI, Low = USB |
| 15:8 | RSV | Reserved | Normally Low |
| 16 | R_CD | Record Carrier Detect | High = No Signal |
| 17 | R_LCK | Record Locked to input stream | High = No lock |
| 18 | R_SYC | Record Sync to input stream | High = No Sync |
| 19 | R_ERR | Record Data errors too high | High = Error Rate |
| 20 | R_PFF | Record Processing Fifo Full | High = Fifo Full |
| 21 | R_PFE | Record Processing Fifo Empty | High = Fifo Empty |
| 22 | R_OFF | Record Output Fifo Full | High = Fifo Full |
| 23 | R_SFF | Record SDRAM Fifo Full | High = Fifo Full |
| 24 | R_RSV | Reserved | Normally Low |
| 25 | P_LCK | Play Locked to output stream | High = No lock |
| 26 | P_BFF | Play Burst Fifo full | High = Fifo Full |
| 27 | P_DFF | Play DVB Output Fifo Full | High = Fifo Full |
| 28 | P_DFE | Play DVB Output Fifo Empty | High = Fifo Empty |
| 29 | P_OFF | Play Output Fifo Full | High = Fifo Error |
| 30 | P_OFE | Play Output Fifo Empty | High = Fifo Empty |
| 31 | P_SFF | Play SDRAM Fifo Full | High = Fifo Full |

- **Firmware Version Table**

| Number | Name | Supports |
|---|---|---|
| 0 | AT200USB_DVB_REC.rbf | ASI + SPI inputs |
| 1 | AT200PCI_DVB_REC.rbf | ASI + SPI inputs |
| 2 | AT200USB_SMPTE_REC. rbf | SMPTE 259 & 125 inputs |
| 3 | AT200PCI_SMPTE_REC.rbf | SMPTE 259 & 125 inputs |
| 4 | AT300USB_DVB_PLAY. rbf | ASI + SPI outputs |
| 5 | AT300PCI_DVB_PLAY.rbf | ASI + SPI outputs |
| 6 | AT300USB_SMPTE_PLAY. rbf | SMPTE 259 & 125 outputs |
| 7 | AT300PCI_SMPTE_PLAY.rbf | SMPTE 259 & 125 outputs |
| 8 | AT400USB_DVB_REC. rbf | ASI + SPI inputs & Pass Through |
| 9 | AT400PCI_DVB_REC.rbf | ASI + SPI inputs & Pass Through |
| 10 | AT400USB_SMPTE_REC. rbf | SMPTE 259 & 125 inputs & Pass Through |
| 11 | AT400PCI_SMPTE_REC.rbf | SMPTE 259 & 125 inputs & Pass Through |
| 12 | AT400USB_DVB_PLAY. rbf | ASI + SPI outputs |
| 13 | AT400PCI_DVB_PLAY.rbf | ASI + SPI outputs |
| 14 | AT400USB_SMPTE_PLAY. rbf | SMPTE 259 & 125 outputs |
| 15 | AT400PCI_SMPTE_PLAY.rbf | SMPTE 259 & 125 outputs |
| 16 | AT500USB. rbf | Two Audio outputs |
| 17 | AT500PCI.rbf | Two Audio outputs |
| 18 | AT600USB_DVB_REC. rbf | DVB-S input + DVB-ASI & SPI output |
| 19 | AT600PCI_DVB_REC.rbf | DVB-S input + DVB-ASI & SPI output |
| 20 | AT600USB_DVB_PLAY. rbf | Play DVB |
| 21 | AT600PCI_DVB_PLAY.rbf | Play DVB |
| 22 | AT700USB_DVB_REC. rbf | DVB-C input + DVB-ASI & SPI output |
| 23 | AT700PCI_DVB_REC.rbf | DVB-C input + DVB-ASI & SPI output |
| 24 | AT700USB_DVB_PLAY. rbf | Play DVB |
| 25 | AT700PCI_DVB_PLAY.rbf | Play DVB |
| 26 | AT800USB_DVB_REC. rbf | DVB-T input + DVB-ASI & SPI output |
| 27 | AT800PCI_DVB_REC.rbf | DVB-T input + DVB-ASI & SPI output |
| 28 | AT800USB_DVB_PLAY. rbf | Play DVB |
| 29 | AT800PCI_DVB_PLAY.rbf | Play DVB |
| 30 | TBD | |
| 31 | TBD | |

- **Data Error Registers (Monitor)**

Data Error Registers    Address = H"10"

| Bit | Name | Function | Level |
|---|---|---|---|
| 31:0 | Data Err | Data error counter | |

This register counts up for every error. There is no reset function. The software must make adjustment for the initial value and the counter roll over.

- **Sync Error Registers (Monitor)**

Sync Error Registers    Address = H”14”

| Bit | Name | Function | Level |
|---|---|---|---|
| 31:0 | Sync Err | Sync error counter | |

This register counts up for every error. There is no reset function. The software must make adjustment for the initial value and the counter roll over.

- **Packet pulse counter Registers (Monitor)**

Packet pulse counter   Address = H”18”

| Bit | Name | Function | Level |
|---|---|---|---|
| 31:0 | Pck Pulse | Packet pulse counter | |

The time of 100 packets is measured and presented. In USB mode, the reference clock is 48MHz, in PCI mode 54MHz. Note that the length of transport packets must be taken into account. Since time tamping is not in the original transport stream, it is not used in the calculation.

- **Available Audio Registers (AT500)**

Available Audio channels   Address = H”1C”

| Bit | Name | Function | Level |
|---|---|---|---|
| 31:0 | Audio Ava | One Bit for each of 16 channels | 1 = available |

- **Selected Audio Registers (AT500)**

Selected Audio channels Output1   Address = H”20”

| Bit | Name | Function | Level |
|---|---|---|---|
| 31:0 | Audio Sel | Bits 31:16 for Right<br>Bits 15:0 for Left | High = active , maximum of two |

Selected Audio channels Output2    Address = H”24”

| Bit | Name | Function | Level |
|---|---|---|---|
| 31:0 | Audio Sel | Bits 31:16 for Right<br>Bits 15:0  for Left | High = active, maximum of two |

- **PCI Data transfer**

Play_FIFO    Address = H”80”

| Bit | Name | Function | Level |
|---|---|---|---|
| 31:0 | InData | Writes into on device FIFO | |

Record_FIFO    Address = H”80”

| Bit | Name | Function | Level |
|---|---|---|---|
| 31:0 | OutData | Reads from on device FIFO | |

- **Tuner Access**

Tuner Data Write    Address = H"90"

| Bit | Name | Function | Level |
|---|---|---|---|
| 31:0 | TunerWData | Tuner Write Data | |

Tuner Communication Control    Address = H"94"

| Bit | Name | Function | Level |
|---|---|---|---|
| 8:0 | TunerIdx | Command Index Number | |
| 11:9 | DataBytes | Number of bytes for Read/Write | 1 to 4 (5 to 7 are invalid) |
| 12 | R/W | Read / Write Action Indicator | 1 = Read, 0 = Write |
| 15:13 | TunerFunc | Device Select | "0"=PLL, "1" = Satellite, "2"= Cable, "3" = Terrestrial |
| 31:16 | Reserved | | |

Tuner Communication Status    Address = H"98"

| Bit | Name | Function | Level |
|---|---|---|---|
| 0 | Busy | Communication in progress | HIGH = Busy |
| 1 | Slave Nack | Error on Slave communication | HIGH = Slave Error |
| 2 | Data Nack | Error on Data communication | HIGH = Data Error |
| 7:4 | Reserved | | Normally Low |

Tuner Read Data    Address = H"99"    ***Note:* Not DWORD aligned!**

| Bit | Name | Function | Level |
|---|---|---|---|
| 31:0 | TunerRData | Tuner Read Data | |

- **PID Table**

The PID table allows inclusion or exclusion of PID's from the incoming transport stream. A value of zero performs no action. The initial value is zero. Note that the addressing is per 16-bit.

PID Table
Address = H"C0" through H"FE"

| Bit | Name | Function | Level |
|---|---|---|---|
| 12:0 | PID | PID number | 0 = not used |
| 15:13 | Reserved | | |

# REGISTER DESCRIPTIONS

- **Record_Config Register** Used by the following devices: AT200, AT400, AT500.

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| FEN | IIO | LRC | LEN | PAR | SER | SMP | DVB |

Address: H"00"
Type: Write/Read
Reset: xx
Description: This is the 8 LSB of the record configuration register.
The Record configuration register holds all the values needed for correct setting of the hardware resources of input section, including the functions of the GS9060 deserializer device.

[0] **DVB mode** Applicable in all modes

[1] **SMPTE mode** Applicable in all modes
Bit 1 (SMP) in conjunction with Bit 0 (DVB), configure the GS9060 to operate in SMPTE, DVB or Data through (RAW DATA) mode.

| | | |
|---|---|---|
| DVB mode | [0] = HIGH, | [1]= LOW |
| SMPTE mode | [0] = LOW, | [1]= HIGH |
| RAW Datamode | [0] = LOW, | [1]= LOW |

[2] **Select the Serial input** Applicable in all modes.
When set HIGH the serial input is selected as the default input for recoding, loop through or pass through regardless of the status of Bit3 (PAR).

[3] **Select the Parallel input** Applicable in all modes.
When set HIGH and Bit2 (SER) is set LOW, the Parallel input is selected as input for recording, loop through or pass through.

[4] **Loop through Enable** Applicable in all modes.
When set HIGH, the serial loop through is enabled.

[5] **Input Loop through re-clock enable** Applicable in all modes.
When set HIGH, the serial loop through output will be a reclocked version of the serial input signal regardless of the mode of operation (SMPTE, DVB-ASI or RAW-Data)
When set LOW, the loop through ouput will be a buffered version of the input signal in all modes.

[6] **Input I/O Processing** Applicable in SMPTE mode of operation only. Ignored in other modes.
Used to enable or disable I/O processing of GS9060 device.
When set HIGH the following I/O processing features of the device are enabled:
- EDH CRC Error Correction
- ANC Data Checksum Correction
- TRS Error Correction
- Illegal Code Remapping

[7] **Fly Wheel Enable** Applicable in SMPTE mode of operation only. Ignored in other modes.
Used to enable or disable the noise immune flywheel of the GS9060 device. The internal flywheel is used in extraction and generation of TRS timing signals. When set LOW, the internal flywheel is disabled and TRS correction and insertion is unavailable.

- **Record_Config Register** Used by the following devices: AT200, AT400, AT500

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| RSV | RSV | RSV | MUT2 | MUT1 | TEX | PID | ETS |

Address: H"01"
Type: Write/Read
Reset: xx
Description: This is the bits 8 to 15 of the record configuration register.

[8] **Enable Time Stamping** Applicable in DVB mode. Ignored in other modes.
When set high the incoming transport stream is time stamped.
The time stamp is derived from the master clock (48MHz for USB version and 54MHz for PCI version).
On arrival of the 11th byte of the transport stream, the PCR byte, the content of a 32 bit free running counter is taken as the time stamp.
The 32 bit time stamp value is then added to the end of the transport stream.
The application then has to be aware of the fact that the last four bytes of the transport streams are the time stamp value not pay load.

[9] **Enable PID filtering** Applicable in DVB mode. Ignored in other modes.
When enabled the hardware will filter out or pass through the selected programs from the PID table.
The PID table consists of 32 registers. As a result maximum of 32 PIDs could be entred into the PID table.
When the PID table contains a program which is not available in the transport stream, it is ignored by the PID filtering program.

[10] **PID Table Exclusive** Applicable in DVB mode. Ignored in other modes.
In order to make the PID filtering more efficient, the PID filter could contain the programs which are to be filtered out or the programs which are to be passed through.
When there are only a few program which the user requires to pass through, (INCLUDED) then it is more efficient to enter these programs into the PID table and set TEX bit LOW indicating the PID table an Inclusive one.
When there are more programs to be passedthrought and only a few to be filtered out, (EXCLUDED) then only these programs are entred into the PID table and the TEX bit is set HIGH. In this way the PID filtering is much faster and does not require a large table.

[11] **Mute the Audio1** Applicable for AT500 Audio de-embedder otherwise ignored.
The AT500 Audio de-embedder device is capable of de-embedding two pairs of audio channals.
The de-embedded audio channals are avialable on two audio outputs.
The Audio1 output could be muted by setting this bit HIGH.

[12] **Mute the Audio2** Applicable for AT500 Audio de-embedder otherwise ignored.
The Audio2 output could be muted by setting this bit HIGH.

[15:13] **Reserved** These bits are reserved for futhure development and are not used in the current version.

- **Record_Config Register** Used by the following devices: AT 200, AT 400, AT500.

| RSV [31:24] | RSV [23:16] |
|---|---|

Address: H"02" and H"03"
Type: Write/Read
Reset: xx
Description: This is the bits 16 to 23 of the record configuration register.

[31:16] **Reserved** These bits are reserved for futhure development and are not used in the current version.

- **Play_Config Register** Used by the following devices: AT300, AT400, AT600, AT700, AT800.

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| RAW | 188+16 | 204 | 188 | PAR | SER | SMP | DVB |

Address: H"04"
Type: Write/Read
Reset: xx
Description: This is the 8 LSB of the Play configuration register.
The Play configuration register holds all the values needed for correct setting of the hardware resources of output section, including the functions of the GS9062 serializer device.

[0] **DVB mode** Applicable in all modes.

[1] **SMPTE mode** Applicable in all modes.
Bit 1 (SMP) in conjunction with Bit 0 (DVB), configure the GS9062 to operate in SMPTE, DVB or Data through (RAW DATA) mode.

| | | |
|---|---|---|
| DVB mode | [0] = HIGH, | [1] = LOW |
| SMPTE mode | [0] = LOW, | [1] = HIGH |
| RAW Data mode | [0] = LOW, | [1] = LOW |

[2] **Enable Serial output** Applicable in all modes.
When set HIGH the serial output is enabled

[3] **Enable Parallel output** Applicable in all modes.
When set HIGH the parallel output is enabled.

[4] **188 Bytes Transport Packet Size**
When set HIGH the transmitted transport packet size of 188 is selected.
This bit is set by the application when a transport stream file is played back.
The application calculates the bit rate and the packet size from the transport stream files and sets the correct packet size and bit rate.

[5] **204 Byte Transport Packet Size** Applicable in DVB mode.
When set HIGH the transmitted transport packet size of 204 is selected.
This bit set by the application when a transport stream file is played back.

[6] **Add 16 bytes to a 188 byte Transport Packet** Applicable in DVB mode.
When this bit and bit4 are both HIGH, 16 bytes are added to the end of the 188 byte transport packet pay load hence making a 204 byte transport stream.
This function is usefull for the designers of reciver equipments to test if the system under development could handel both 188 and 204 packet sizes.

[7] **Transmit RAW data** Applicable in all modes.
When HIGH the tranmitted data is regarded as NONE-DVB data, as such there are no attampts made to comply to DVB standards.
This mode is of interset to the designer of reciver equipments to test their systems for detection of streams which are not DVB-ASI complient as well as those applications which only requires a high speed serial data link.

- **Play_Config Register** Used by the following devices: AT300, AT400, AT600, AT700, AT800.

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| IOP | BLK | CTP | NTP | GTP | BURST | EXCLK | CLK27 |

Address: H"05"
Type: Write/Read
Reset: xx
Description: This is bits 8 to 15 of the Play Configuration Register.
The play configuration register holds all the values needed for correct setting of the hardware resources of output section, including the functions of the GS9062 serializer device.

[8] **Select the on device 27MHz clock for transmission of data** Applicable to DVB-ASI mode only.
By default the on device 27MHz clock is the transmission clock.

[9] **Select the External clock for data transmission.** Applicable to all modes.
In applications where there is a need for using an external clock for data transmission, Alitronika devices provide an external input clock.
Setting this bit HIGH will select the external clock instead of the on device clock.

[10] **Transport Stream in burst mode** Applicable to DVB-ASI mode only.
The DVB-ASI standards specify that data is transmitted at a constant bit rate of 270 Mbit/s. Considering the 8 bit data bytes are 8b/10b encoded, this correspond to fixed symbol rate of 27 Msymbol/s.
But the transport bit rate could be much less, special character, K28.5 (Comma) is used as 'Stuffing' to make up the difference. The transport packets may be transmitted as a burst of contiguous bytes or as individual bytes spread out in time. Normally the later used, since spreading out the data in time has less demand on the receiver's input buffer. Alitronika devices use this mode of transmission as default.
By setting the BURST mode HIGH the burst mode could be activated.

[11] **Hardware Generated Transport Stream** Applicable to DVB-ASI mode only.
In the play mode a transport stream could be played back from a file. Alitronika devices with output function could also generate simple transport streams by hardware. This function has the advantage of freeing the device of taking up any processing power from the system while providing a stream for testing purposes. Two types of transport streams could be generated by the hardware.
Setting this bit HIGH enables the hardware generation of transport stream.

[12] **Null Packet Generation** Applicable to DVB-ASI mode only.
When set HIGH, this bit in conjuction with Bit11 (GTP) will enable the harware generation of a Null packet transport stream.

[13] **Counter Packet Generation.** Applicable to DVB-ASI mode only.
Alhtough very usefull for many tests, the Null packet is not sufficient to test the integrity of a DVB-ASI transmission link. In the other hand if a transport packet of say 188 byte size contains bytes which count from 0 to 188, with the byte H"47" replaced by the content of another free running counter, the integrity of the transmission link can easily be varified. When set HIGH, this bit in conjuction with Bit11 (GTP) will enable the harware generation of a Counter packet transport stream.

[14] **BLANKING** Applicable to SMPTE mode only.
Used to enable or disable the input data blanking of the GS9062 serializer device in SMPTE mode of operation. When HIGH the LUMA and CHROMA input data is set to appropriate blanking level. Horizontal and vertical ancillary spaces will also be set to blanking levels. When set LOW, the LUMA and CHROMA input data pass through the device unaltered.

[15] **Enable output I/O Processing** Applicable to SMPTE mode only.
Used to enable or disable I/O processing of GS9062 serializer device in SMPTE mode of operation.
When set HIGH the following I/O processing features of the device are enabled:
- EDH Packet Generation and Insertion
- SMPTE 352M Packet Generation and Insertion
- ANC Data Checksum Calculation and Insertion
- TRS Generation and Insertion
- Illegal Code Remapping

- **Play_Config Register** Used by the following devices: AT300, AT400, AT600, AT700, AT800.

| | | | 19 | 18 | 17 | 16 | 16 |
|---|---|---|---|---|---|---|---|
| RSV | RSV | RSV | CPS | CSP | PPT | SPT | TRS |

Address: H"06"
Type: Write/Read
Reset: xx
Description: This is bits 16 to 23 of the Play configuration register.
The Play configuration register holds all the values needed for correct setting of the hardware resources of output section, including the functions of the GS9062 serializer device.

[16] **Detect TRS** Applicable to SMPTE mode only.
Used to select the timing mode of GS9062 serializer device in SMPTE mode of operation. When set HIGH, the device will lock the internal flywheel to the embedded TRS timing signals in the parallel input data.

[16] **Enable Serial Pass Through Output** Applicable in all modes.
Setting this bit HIGH would pass through the serial input to the serial output.
This function could be used to generate more signal from a single signal.

[17] **Enable Parallel Pass Through Output.** Applicable in all modes.
Setting this bit HIGH would pass through the parallel input to the parallel output.
This function is used in case the parallel data is needed simultaosly to be used by another equipment.

[18] **Enable Serial to parallel conversion** Applicable in all modes.
When HIGH the hardware would convert the input serial data into a parallel data transmitted via the LVDS/ECL output port provided the bit rate of the input data does not exced the maximum bit rate allowed by the parallel data transmission.

[19] **Enable Parallel to Serial conversion** Applicable in all modes.
When HIGH the parallel input data is converted to serial data.

- **Play_Config Register** Used by the following devices: AT300, AT400, AT600, AT700, AT800.

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| RSV | RSV | RSV | RSV | RSV | RSV | RSV | RSV |

Address: H"07"
Type: Write/Read
Reset: xx
Description: This is bits 24 to 31 of the Play configuration register.
The Play configuration register holds all the values needed for correct setting of the hardware resources of output section, including the functions of the GS9062 serializer device.

[31:24] **RSV: Reserved** These bits are reserved for future development and are not use in the current design.

- **Bit Rate Register** Used by the following devices: AT300, AT400, AT600, AT700, AT800.

| BitRate[31:24] | BitRate[23:16] | BitRate[15:8] | BitRate[7:0] |
|---|---|---|---|

Address: H"08", H"09", H"0A" and H"0B"
Type: Write/Read
Reset: xx
Description: This 32 bit register contains the output bit rate of the transport stream.

[31:0] **Transport stream Output Bit rate** Applicable to DVB mode only and is not used in any other mode.

- **Record_Status Register** Used by the following devices: All devices.

| 7 | 6 | 5 | 4 | 3 | 2 | 1 | 0 |
|---|---|---|---|---|---|---|---|
| BUS | VER4 | VER3 | VER2 | VER1 | VER0 | R_TPE | R_TPS |

Address: H"0C"
Type: Read
Reset: xx
Description: This register contains the status of the input section.

[0] **R_TPS: Input Transport Packet size** Applicable to DVB mode, Ignored by othere modes.
The packet size of the input transport stream is determined by the hardware syncronizarion mechanisem.
When this bit set HIGH the input transport packet size is 188 and when is set LOW the input transport packet
size 204.

[0] **R_TPS: Input Transport Packet size Error** Applicable to DVB mode, Ignored by othere modes.
The input transport stream should be 188 or 204. If due some errors in the received input stream, the packet size is not 188 or 204, this bit is set HIGH by the hardware syncronization mechanisem to indicate a packet size error condition.

[6:2] **Firmware Version** Applicable to all devices in all modes.
These bits identify the current version of the firmware installed in the main controller of Alitronika devices. A list of all the firmware versions are shown in the 'Firmware Version Table'.

[7] **Bus type indication** Applicable to all devices in all modes.
Alitronika devices are based on PCI or USB bus versions. This bit could be regarded as the extension of the firmware version. This bit set HIGH for PCI version of the device and set LOW for USB version.

- **Play_Status Register** Used by the following devices: all devices.

| 15 | 14 | 13 | 12 | 11 | 10 | 9 | 8 |
|---|---|---|---|---|---|---|---|
| RSV | RSV | RSV | RSV | RSV | RSV | RSV | RSV |

Address: H"0D"
Type: Read
Reset: xx
Description: This register contains the status of the input section.

[15:8] **Reserved** Reserved. Not used in the current version of the design.

- **Record_Error Register** Used by the following devices: AT200, AT400, AT500, AT600, AT700, AT800.

| 23 | 22 | 21 | 20 | 19 | 18 | 17 | 16 |
|---|---|---|---|---|---|---|---|
| R_SFF | R_OFF | R_PFE | R_PFF | R_ERR | R_SYC | R_LCK | R_CD |

Address: H"0E"
Type: Read
Reset: xx
Description: This register contains the error flags generated by the input section.
Since these bits are all error flages they are active high
to indicate the error condition.

[16] Record section Carrier Detect Applicable in all modes.
This bit is generated by the cable equlizer. As long as this bit is set HIGH there is no carrier detected by the input circuitry.

[17] Record section Locked to input data Applicable in all modes.
This flag is generated by the GS9060 input device. This bit will be LOW whenever the device has correctly received and locked to SMPTE compliant data in SMPTE mode or DVB-ASI compliant data in DVB-ASI mode or when the reclocker has achieved lock in RAW data (Data Through) mode.

[18] Record section Syncronized to the input Stream Applicable in DVB-ASI mode.
In DVB-ASI mode the syncronization to the incoming data stream is achived by the main controller.
The syncronizer has to detect the transport stream sync byte, H"47", for every packet, 188 or 204 byte apart, depending on the packet size. When this is the case Bit2, R_SYC, is set LOW. When Syncronization is lost this bit is set HIGH.

[19] Incoming data error Applicable in DVB-ASI mode.
In every serial link a degree of errors are tolerated, if the number of errors, that is the number of corrupted bytes or bytes which can not be decoded by the DVB-ASI decoder, are too high the link is no loger viable.
The syncronizer in the main controller sets the R_ERR to HIGH whenever this condition is reached.

[20] Record Processing Input Fifo Full (overflow) Applicable in DVB-ASI mode.
The incoming data is processed by the main controller, the input Fifo is used by this setion will set the R_PFF error flag HIGH whenever an overflow condition has occurred.

[21] Record Processing Fifo Input Empty (underflow) Applicable in DVB-ASI mode.
The incoming data is processed by the main controller, the input Fifo is used by this setion will set the R_PFE error flag HIGH whenever an underflow condition has occurred.

[22] Record Processing Output Fifo Full (overflow) Applicable in DVB-ASI mode.
The incoming data is processed by the main controller, the output Fifo is used by this setion will set the R_OFF error flag HIGH whenever an overflow condition has occurred.

{23] Record SDRAM Fifo Full (overflow) Applicable in all modes.
The SDRAM used as Fifo will set the R_SFF error flag HIGH whenever an overflow condition has occurred.

- **Play_Error Register** Used by the following devices: AT300, AT400, AT500, AT600, AT700, AT800.

| 31 | 30 | 29 | 28 | 27 | 26 | 25 | 24 |
|---|---|---|---|---|---|---|---|
| P_SFF | P_OFE | P_OFF | P_DFE | P_DFF | P_BFF | P_LCK | RSV |

Address: H"0F"
Type: Read
Reset: xx
Description: This register contains the error flags generated by the output section.
Since these bits are all error flages they are active high to indicate the error condition.

[24] **Reserved** Applicable in all modes.
Not used in the current design.

[25] **Record section Locked to input data** Applicable in all modes.
This flag is generated by the GS9062 output device. This bit will be LOW whenever the device has correctly received and locked to SMPTE compliant data in SMPTE mode or DVB-ASI compliant data in DVB-ASI mode or when the reclocker has achieved lock in RAW data (Data Through) mode.

[26] **Play Burst Fifo Full (overflow)** Applicable in DVB-ASI mode.
In DVB-ASI mode the out going transport packets could be send in burst mode. The data can enter the main controller in none-burst mode. A Fifo is therefore needed to hold a full packet before transmission. When this fifo detects an overflow the P_BFF flag will be set HIGH.

[27] **Play DVB Fifo Full (overflow)** Applicable in DVB-ASI mode.
The out going DVB data stream is processed by the main controller, the Fifo used by this setion will set the P_DFE error flag HIGH whenever an overflow condition has occurred.

[28] **Play DVB Fifo Empty (underflow)** Applicable in DVB-ASI mode.
The out going DVB data stream is processed by the main controller, the Fifo used by this setion will set the P_DFE error flag HIGH whenever an underflow condition has occurred.

[29] **Play Output Fifo Full (overflow)** Applicable in all modes.
The output Fifo will set the P_OFF error flag HIGH whenever an overflow condition has occurred.

[30] **Play Output Fifo Empty (underflow)** Applicable in all modes.
The output Fifo will set the P_OFE error flag HIGH whenever an underflow condition has occurred.

[31] **Play SDRAM Fifo Full (overflow)** Applicable in all modes.
The SDRAM used as Fifo will set the P_SFF error flag HIGH whenever an overflow condition has occurred.

- **Input Data Error Register** Used by the following devices: AT200, AT400, AT600, AT700, AT800.

| DataErr[31:24] | DataErr[23:16] | DataErr[15:8] | DataErr[7:0] |
|---|---|---|---|

Address: H"10", H"11", H"12" and H"13"
Type: Read
Reset: xx
Description: This 32 bit register contains the number of data bytes errors.
Whenever the GS9060 device, which decodes the DVB-ASI incoming stream, detects an illegal code word, it generates a 'Word Error' flag for the corresponding byte. A free running counter counts the number of errors.
The Data Error Register contains the content of this error counter. The Data error counter counts up for every error. There is no reset and the register may rap around. The application must compensate for these when this function is used to monitor data errors in DVB-ASI mode of operation.

- **Input Sync Error Register** Used by the following devices: AT200, AT400, AT600, AT700, AT800.

| SyncErr[31:24] | SyncErr[23:16] | SyncErr[15:8] | SyncErr[7:0] |
|---|---|---|---|

Address: H"14", H"15", H"16" and H"17"
Type: Read
Reset: xx
Description: This 32 bit register contains the number of Sync Errors. Whenever the the syncroniser which syncronises to the incoming DVB-ASI stream, does not find the sync byte, H"47", at the start of a transport packet, it increaments a free running 'Sync counter'. The Sync Error Register contains the content of this counter. The Sync error counter counts up for every error. There is no reset and the register may rap around. The application must compensate for these when this function is used to monitor sync errors in DVB-ASI mode of operation.

- **Packet pulse counter Register** Used by the following devices: AT200, AT400, AT600, AT700, AT800.

| Pck Pulse[31:24] | Pck Pulse[23:16] | Pck Pulse[15:8] | Pck Pulse[7:0] |
|---|---|---|---|

Address: H"18", H"19", H"1A" and H"1B"
Type: Read
Reset: xx
Description: This 32 bit register is used to estimate the bit rate of an incoming transport stream.
A free running counter is started at the beginning of a packet and stoped at the end of 100th packets, hence the number of clock pulses per 100 packets is obtained.The application uses this number together with the Packet Size information (i.e 188 or 204 byte packets) and the clock frequency, 48 MHz for USB and 54 MHz for PCI to calculate the bit rate.

- **Available Audio Channel Register** Used by the following devices: AT500.

| AudioAva[31:24] | AudioAva[23:16] | AudioAva[15:8] | AudioAva[7:0] |
|---|---|---|---|

Address: H"1C", H"1D", H"1E" and H"1F"
Type: Read
Reset: xx
Description: a 32 bit register which contains the audio channels available.
There can be as many as 16 audio channels embedded in a SDI stream. The main controller, detects which of these audio channels are present in the input SDI stream. The 16 LSB are used for SDI input1 and the 16 MSB are used by the second SDI input.

- **Selected Audio channels for Audio Output1 Register** Used by the following devices: AT500.

| Right Audio [31:16] | Left Audio [15:0] |
|---|---|

Address: H"20", H"21", H"22" and H"23"
Type: Write/ Read
Reset: xx
Description: a 32 bit register which contains the selected audio channels for the output1. Bits 0 to 15 are used for the left and Bits 16 to 31 for the right by setting the corresponding bit HIGH. So if Audio Channel 3 is avaliable and is then selected for left audio, then bit 2 has to be set HIGH and all the other bits set LOW.
There can be as many as 16 audio channels embedded in a SDI stream. The main controller, detects which of these audio channels are present in the input SDI stream. Two audio channels can then be selected for de-embedding for each Audio output.

- **Selected Audio channels for Audio Output2 Register** Used by the following devices: AT500.

| Right Audio [31:16] | Left Audio [15:0] |
|---|---|

Address: H"24", H"25", H"26" and H"27"
Type: Write/ Read
Reset: xx
Description: a 32 bit register which contains the selected audio channels for the output1. Bits 0 to 15 are used for the left and Bits 16 to 31 for the right by setting the corresponding bit HIGH. So if Audio Channel 3 is avaliable and is then selected for left audio, then bit 2 has to be set HIGH and all the other bits set LOW.
There can be as many as 16 audio channels embedded in a SDI stream.
The main controller, detects which of these audio channels are present in the input SDI stream.
Two audio channels can then be selected for de-embedding for each Audio output.

- **PCI Data transfer register** Used by the following devices: All PCI version of devices.

| Data [31:24] | Data [23:16] | Data [15:8] | Data [7:0] |
|---|---|---|---|

Address: H"80", H"81", H"82" and H"83"
Type: Write/ Read
Reset: xx
Description: a 32 bit register which is used for the data transfere between the main controller and the PCI device.

- **Tuner Data transfer register** Used by the following devices: AT600, AT700, AT800.

| TunerData [31:24] | TunerData [23:16] | TunerData [15:8] | TunerData [7:0] |
|---|---|---|---|

Address: H"90", H"91", H"92" and H"93"
Type: Write/ Read
Reset: xx
Description: a 32 bit register which is used for the data transfere between the main controller and the RF tuner/ demodulator devices.

- **Tuner Communication control Register** Used by the following devices: AT600, AT700, AT800.

| RSV [31:16] | TunerType [15:13] | R/W [12] | DataLen [11:9] | TunerIdx [8:0] |
|---|---|---|---|---|

Address: H"94", H"95", H"96" and H"97"
Type: Write/ Read
Reset: xx
Description: a 32 bit register which contains the control for accessing the functions of the DVB-S NIM device.

[8:0] Command index number

[11:9] Number of bytes to write/read
A maximum of four bytes may be written or read by each Write or Read operation.

[12] Read/ write action indicator
HIGH for Read Operation and LOW for Write operation

[15:13] Selects tuner type
These two bits select the device for communication
(0) PLL
(1) DVB-S, Satellite
(2) DVB-C, Cable
(3) DVB-T, Terrestrial

[31:16] Reserved
These bits are either used by Alitronika's other devices or reserved for future development and are not used in the current version.

- **Tuner Communication Status Register** Used by the following devices: AT600, AT700, AT800.

| 7:4 | 2 | 1 | 0 |
|---|---|---|---|
| | | | |

Address: H"98"
Type: Read
Reset: xx
Description: a 8 bit register which contains the status of the RF tuner/demodulator device.

[0] Communication in progress
When HIGH this bit indicates the I2C bus is busy.

[1] Error on Slave communication
When HIGH there is a communication error by the I2C bus Slave device, the DVB-S NIM device

[2] Error on Data communication
When HIGH there is a data communication error by the I2C bus.

[7:3] Reserved
These bits are either used by Alitronika's other devices or reserved for future development and are not used in the current version.

- **TunerReadData register** Used by the following devices: AT600, AT700, AT800.

| TunerRData [31:24] | TunerRData [23:16] | TunerRData [15:8] | TunerRData [7:0] |
|---|---|---|---|

Address: H"99", H"9A", H"9B" and H"9C"
Type: Read
Reset: xx
Description: a 32 bit register which is used only by the USB version of the devices to read registers of the demodulator devices.

**Note:** The PCI version uses the **PCI Data transfer register** at address H"90" exclusively to read these registers.

**Write & Read Operation of Tuner Registers**

The tuner is controlled by the FPGA via an I2C interface with the FPGA.
Registers at address H"90" to H"9C" are used by the I2C mater of FPGA.

Write Operation:

In any Write Operation the data is written into the Tuner Data Write register at address H"90" to H"93". Up to four bytes could be written in one Write Operation. The destination register address is written into register H"94". For Write Operations bit 12 of Tuner Communication Control Address, H"95", must be set LOW. The number of bytes to be written are places into bits 11 to 9.
The I2C master starts the Write Operation after register H"95" is written into.

Read Operation:

The Read Operation is carried out in the same maner. The number of byte to be read,
Up to a maximum of four bytes, are written into bit 11:9, DataBytes, of H"95".
Bit 12 is set HIGH. T the I2C master starts the Read Operation starts as soon as register H"95" is witten into.
Next the Busy bit, bit0, of register H"95" must be pulled to complete the Read Operation.
The data is now available in the Tuner Read Data registers, address H"99 to H"9C"

**Example1: A Write Operation** Write H"22" to the register at (H"01") and H"44" to the register at (H"02") of the Tuner.

1. Write TunerWData[7:0] (H"90") = H"22"
2. Write TunerWData[15:8] (H"91") = H"44"
3. Write Tuner[7:0] (H"94") = H"01" (offset address Tuner)
4. Write Tuner[15:8] (H"95") = H"24"
   - DataBytes = 2
   - R/W = 0 (Write Operation)
   - TunerType = 1
5. After the Tuner[15:0] register is written into , the actual I2C write transfer is started by the I2C master .
6. Pull Communication Status register,(H"98"), bit0. When bit0 is zero the read transfer is done.
   If bit1 or bit 2 are high, there was an error during the transfer.

**Example2: A Read Operation**

Read the Tuner registers at the following address H"10", H"11" and H"12"

6. Write Tuner[7:0] (H"94") = H"10" (offset address Tuner)
7. Write Tuner[15:8] (H"95") = H"76"
   - DataByte = 3
   - R/W = 1 (Read Operation )
   - TunerType = 1
1. After the Tuner[15:8] register is written into, the actual I2C read transfer is started by the I2C master.
2. Pull Communication Status register,(H"98"), bit0. When bit0 is zero the read transfer is done.
   If bit1 or bit 2 are high, there was an error during the transfer.
3. Read TunerRData[7:0] (H"99") = Content of H"10"
4. Read TunerRData[15:8] (H"9A") = Content of H"11"
5. Read TunerRData[23:16] (H"9B") = Content of H"12"

- **PID Filter Table** Used by the following devices: AT200, AT400, AT600, AT700, AT800.

| RSV [15:13] | PID number [12:0] |
|---|---|

Address: H"C0" to H"FE"
Type: Write/ Read
Reset: 00
Description: The PID table is only used in DVB mode of all Alitronika devices with an input.
There are 32 registers each 16 bits which contain the Program ID of the programs to be filtered out or passed through depending on the selection of an EXCLUSIVE or INCLUSIVE table. A maximum of 32 PIDs may be entred into the PID table.
When the PID table contains a program which is not available in the transport stream, it is ignored by the PID filtering program. Value of zero also performs no filtering action.

[12:0] **PID value** Applicable in DVB-ASI mode
The Program ID, is a 13 bit number contained in the header of each transport packet.

[15:13] **Reserved**
These bits are either used by Alitronika's other devices or reserved for future development and are not used in the current version.